LAB EXPERIMENTS 1-10

NAME: S.THARUN KAMALESH

REG NO:-192473004

Experiment 1 – Select Distinct Department IDs

AIM

To write a Pandas program to select and display the distinct Department IDs from the employee department data.

ALGORITHM

1. Import the Pandas library.
2. Create the department data as a DataFrame.
3. Select the DEPARTMENT_ID column.
4. Use unique() to obtain distinct Department IDs.
5. Display the result.

CODE

```
import pandas as pd

data = {
    "DEPARTMENT_ID": [
        10, 20, 30, 40, 50, 60, 70, 80, 90,
        100, 110, 120, 130, 140, 150, 160,
        170, 180, 190, 200, 210, 220, 230,
        240, 250, 260, 270
    ]
}

df = pd.DataFrame(data)

print("Distinct Department IDs:")
print(df["DEPARTMENT_ID"].unique())
```

OUTPUT

```
Distinct Department IDs:
[ 10  20  30  40  50  60  70  80  90 100 110 120 130 140 150 160 170 180
 190 200 210 220 230 240 250 260 270]
```

Experiment 2 – Employees Who Have Done Two or More Jobs

AIM

To identify the employees who have performed two or more jobs using Pandas groupby and size operations.

ALGORITHM

1. Import Pandas.

2. Create the job-history data as a DataFrame.

3. Group the records using EMPLOYEE_ID.

4. Count the number of job records for each employee.

5. Select employees whose count is two or more.

6. Display their employee IDs.

CODE

```
import pandas as pd

data = {
    "EMPLOYEE_ID": [
        102, 101, 101, 201, 114,
        122, 200, 176, 176, 200
    ],
    "JOB_ID": [
        "IT_PROG", "AC_ACCOUNT", "AC_MGR",
        "MK_REP", "ST_CLERK", "ST_CLERK",
        "AD_ASST", "SA_REP", "SA_MAN", "AC_ACCOUNT"
    ]
}

df = pd.DataFrame(data)

job_count = df.groupby("EMPLOYEE_ID").size()

result = job_count[job_count >= 2]

print("Employees who have done two or more jobs:")
print(result.index.tolist())
```

OUTPUT

```
Employees who have done two or more jobs:
[101, 176, 200]
```

Experiment 3 – Display Job Details in Descending Order

AIM

To display job details in descending order of JOB_TITLE.

ALGORITHM

1. Import Pandas.

2. Create the job details DataFrame.

3. Store JOB_ID, JOB_TITLE, MIN_SALARY and MAX_SALARY.

4. Sort the DataFrame using JOB_TITLE.

5. Set ascending=False for descending order.

6. Display the sorted DataFrame.

CODE

```
import pandas as pd

data = {
    "JOB_ID": [
        "AD_PRES", "AD_VP", "AD_ASST", "FI_MGR",
        "FI_ACCOUNT", "AC_MGR", "AC_ACCOUNT",
        "SA_MAN", "SA_REP", "PU_MAN", "PU_CLERK",
        "ST_MAN", "ST_CLERK", "SH_CLERK", "IT_PROG",
        "MK_MAN", "MK_REP", "HR_REP", "PR_REP"
    ],
    "JOB_TITLE": [
        "President",
        "Administration Vice President",
        "Administration Assistant",
        "Finance Manager",
        "Accountant",
        "Accounting Manager",
        "Public Accountant",
        "Sales Manager",
        "Sales Representative",
        "Purchasing Manager",
        "Purchasing Clerk",
        "Stock Manager",
        "Stock Clerk",
        "Shipping Clerk",
        "Programmer",
        "Marketing Manager",
        "Marketing Representative",
        "Human Resources Representative",
        "Public Relations Representative"
    ],
    "MIN_SALARY": [
        20080,15000,3000,8200,4200,8200,4200,
        10000,6000,8000,2500,5500,2008,2500,4000,
        9000,4000,4000,4500
    ],
    "MAX_SALARY": [
        40000,30000,6000,16000,9000,16000,9000,
        20080,12008,15000,5500,8500,5000,5500,10000,
        15000,9000,9000,10500
    ]
}

df = pd.DataFrame(data)

result = df.sort_values(by="JOB_TITLE", ascending=False)

print(result)
```

OUTPUT

	JOB_ID	JOB_TITLE	MIN_SALARY	MAX_SALARY
11	ST_MAN	Stock Manager	5500	8500
12	ST_CLERK	Stock Clerk	2008	5000
13	SH_CLERK	Shipping Clerk	2500	5500
8	SA_REP	Sales Representative	6000	12008
7	SA_MAN	Sales Manager	10000	20080
9	PU_MAN	Purchasing Manager	8000	15000
10	PU_CLERK	Purchasing Clerk	2500	5500
18	PR_REP	Public Relations Representative	4500	10500
6	AC_ACCOUNT	Public Accountant	4200	9000
14	IT_PROG	Programmer	4000	10000
0	AD_PRES	President	20080	40000
16	MK_REP	Marketing Representative	4000	9000
15	MK_MAN	Marketing Manager	9000	15000
17	HR_REP	Human Resources Representative	4000	9000
3	FI_MGR	Finance Manager	8200	16000
1	AD_VP	Administration Vice President	15000	30000
2	AD_ASST	Administration Assistant	3000	6000
5	AC_MGR	Accounting Manager	8200	16000
4	FI_ACCOUNT	Accountant	4200	9000

Experiment 4 – Alphabet Stock Price Line Plot

AIM

To create a line plot showing Alphabet stock prices for the given dates.

ALGORITHM

1. Import Pandas and Matplotlib.

2. Create the stock-date and closing-price data.

3. Convert Date into datetime format.

4. Plot Date on the X-axis and Close price on the Y-axis.

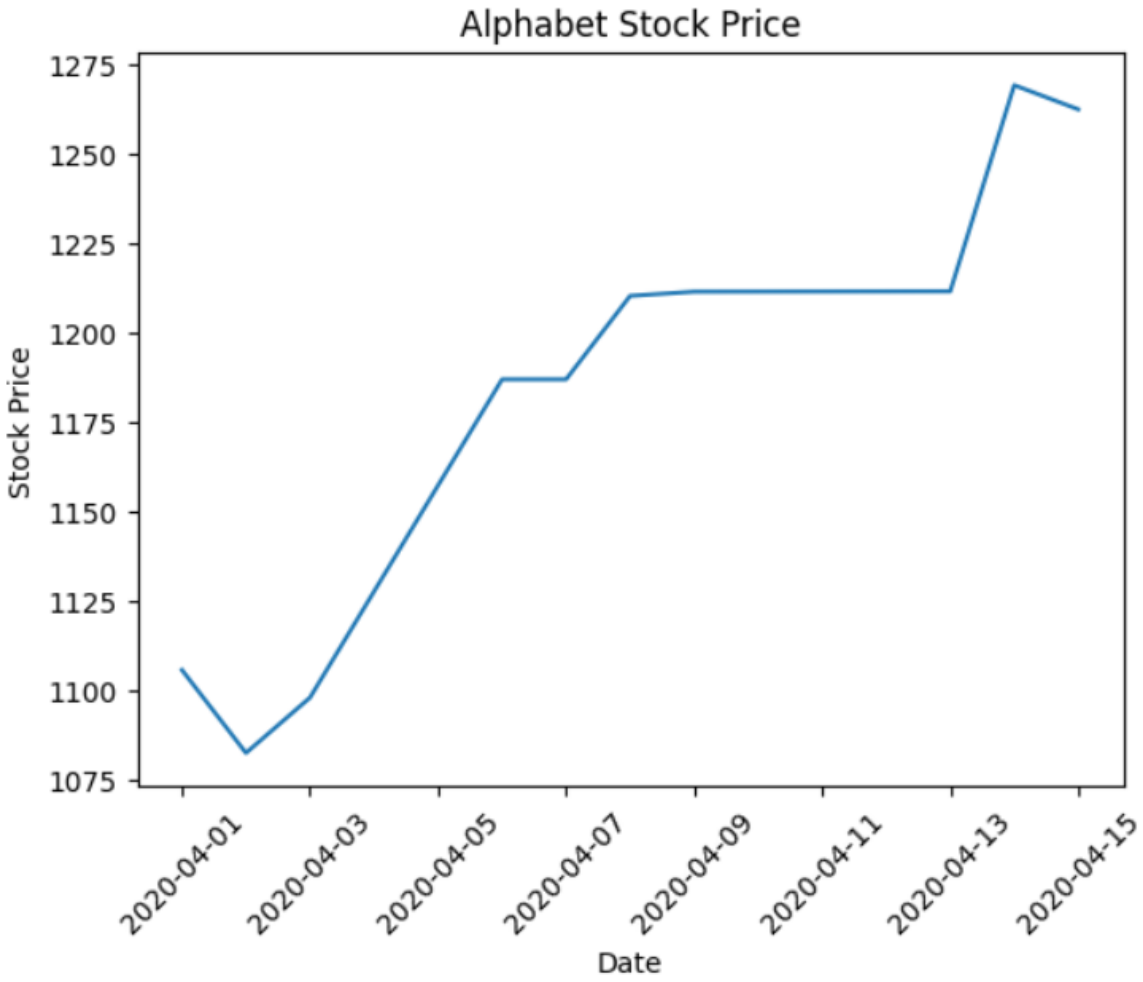
5. Add title and axis labels.

6. Display the line plot.

CODE

```
import pandas as pd
import matplotlib.pyplot as plt

data = {
    "Date": [
        "01-04-2020","02-04-2020","03-04-2020",
        "06-04-2020","07-04-2020","08-04-2020",
        "09-04-2020","13-04-2020","14-04-2020",
        "15-04-2020"
    ],
    "Close": [
        1105.62,1082.39,1097.88,1186.92,1186.92,
        1210.28,1211.45,1211.54,1269.23,1262.47
    ]
}

df = pd.DataFrame(data)
df["Date"] = pd.to_datetime(df["Date"], dayfirst=True)

plt.plot(df["Date"], df["Close"])
plt.xlabel("Date")
plt.ylabel("Stock Price")
plt.title("Alphabet Stock Price")
plt.xticks(rotation=45)
plt.show()
```

OUTPUT


Alphabet Stock Price
Stock Price
1275
1250
1225
1200
1175
1150
1125
1100
1075
2020-04-01
2020-04-03
2020-04-05
2020-04-07
2020-04-09
2020-04-11
2020-04-13
2020-04-15
Date

Experiment 5 – Alphabet Trading Volume Bar Plot

AIM

To create a bar plot showing Alphabet trading volume for the given dates.

ALGORITHM

1. Import Pandas and Matplotlib.

2. Create the Date and Volume data.

3. Convert Date into datetime format.

4. Plot Date against trading Volume using a bar chart.

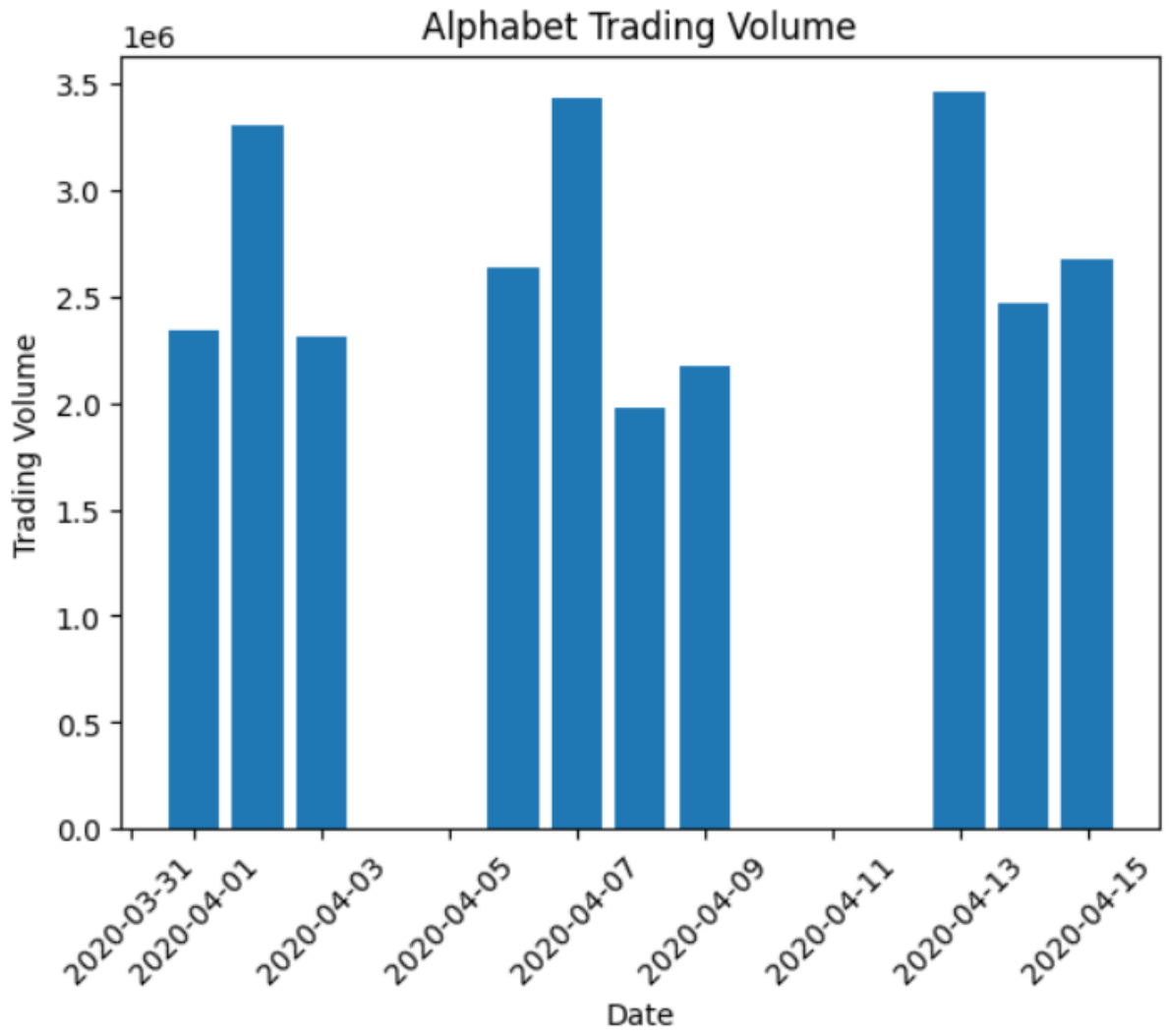
5. Add title and axis labels.

6. Display the graph.

CODE

```
import pandas as pd
import matplotlib.pyplot as plt

data = {
    "Date": [
        "01-04-2020","02-04-2020","03-04-2020",
        "06-04-2020","07-04-2020","08-04-2020",
        "09-04-2020","13-04-2020","14-04-2020",
        "15-04-2020"
    ],
    "Volume": [
        2343100,3304800,2313400,2634700,3432700,
        1975100,2175400,3458900,2470400,2671700
    ]
}

df = pd.DataFrame(data)
df["Date"] = pd.to_datetime(df["Date"], dayfirst=True)

plt.bar(df["Date"], df["Volume"])
plt.xlabel("Date")
plt.ylabel("Trading Volume")
plt.title("Alphabet Trading Volume")
plt.xticks(rotation=45)
plt.show()
```

OUTPUT


Alphabet Trading Volume
1e6
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
Trading Volume
2020-03-31
2020-04-01
2020-04-03
2020-04-05
2020-04-07
2020-04-09
2020-04-11
2020-04-13
2020-04-15
Date

Experiment 6 – Trading Volume vs Stock Price Scatter Plot

AIM

To create a scatter plot to visualize the relationship between trading volume and Alphabet stock price.

ALGORITHM

1. Import Pandas and Matplotlib.

2. Create the stock price and volume data.

3. Store the data in a DataFrame.

4. Use Volume on the X-axis and Close price on the Y-axis.

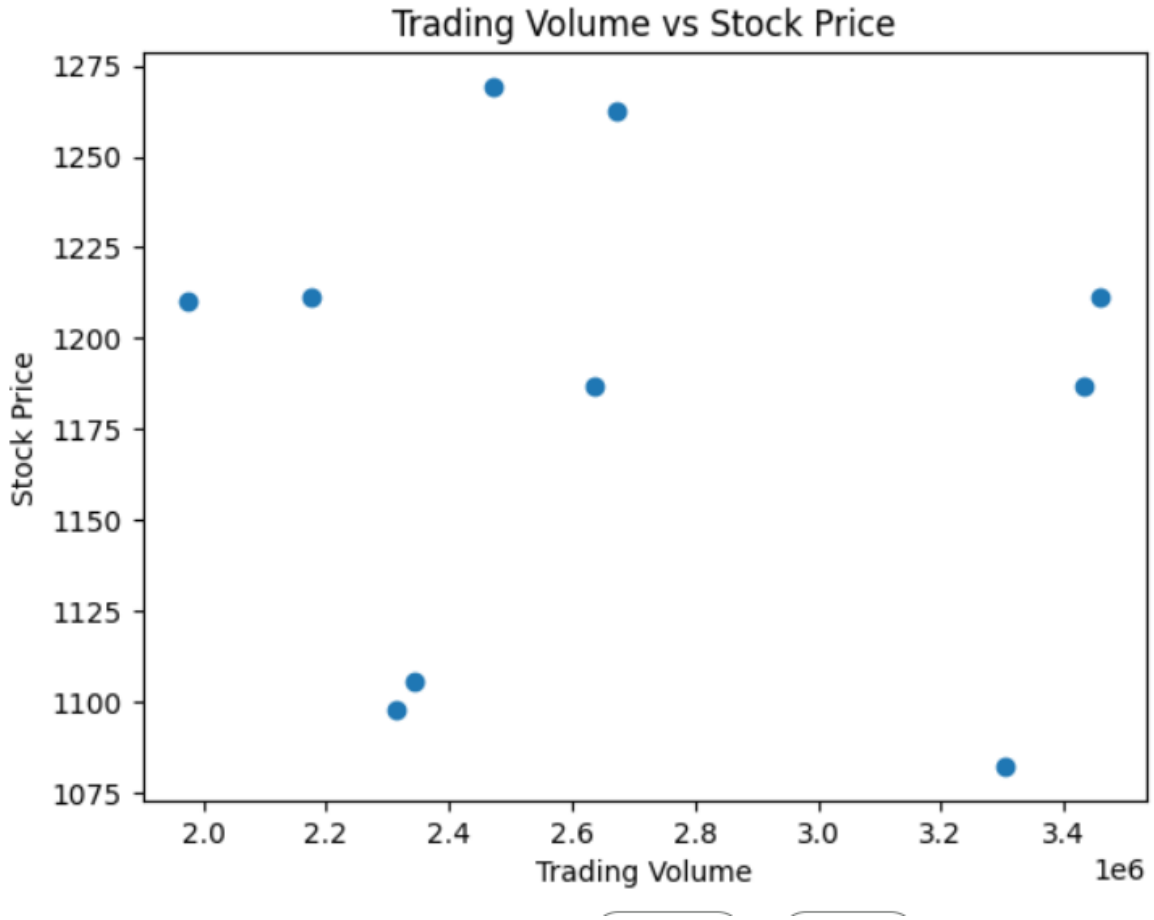
5. Create a scatter plot.

6. Add title and labels and display the plot.

CODE

```
import pandas as pd
import matplotlib.pyplot as plt

data = {
    "Date": [
        "01-04-2020","02-04-2020","03-04-2020",
        "06-04-2020","07-04-2020","08-04-2020",
        "09-04-2020","13-04-2020","14-04-2020",
        "15-04-2020"
    ],
    "Close": [
        1105.62,1082.39,1097.88,1186.92,1186.92,
        1210.28,1211.45,1211.54,1269.23,1262.47
    ],
    "Volume": [
        2343100,3304800,2313400,2634700,3432700,
        1975100,2175400,3458900,2470400,2671700
    ]
}

df = pd.DataFrame(data)

plt.scatter(df["Volume"], df["Close"])
plt.xlabel("Trading Volume")
plt.ylabel("Stock Price")
plt.title("Trading Volume vs Stock Price")
plt.show()
```

OUTPUT


Trading Volume vs Stock Price
Stock Price
1275
1250
1225
1200
1175
1150
1125
1100
1075
2.0
2.2
2.4
2.6
2.8
3.0
3.2
3.4
Trading Volume
1e6

Experiment 7 – Pivot Table for Maximum and Minimum Sale Amount

AIM

To create a Pandas pivot table and find the maximum and minimum sale amount for each item.

ALGORITHM

1. Import Pandas.

2. Create the Sales_data DataFrame.

3. Select Item and Sale_amt.

4. Create a pivot table using Item as the index.

5. Apply max and min aggregation to Sale_amt.

6. Display the pivot table.

CODE

```
import pandas as pd

data = {
    "Item": [
        "Television","Home Theater","Television","Cell Phone",
        "Television","Home Theater","Television","Television",
        "Television","Home Theater","Television","Home Theater",
        "Home Theater","Television","Desk","Video Games",
        "Home Theater","Cell Phone"
    ],
    "Units": [
        95,50,36,27,56,60,75,90,32,60,90,29,81,35,2,16,28,64
    ],
    "Sale_amt": [
        113810,25000,43128,6075,67088,30000,89850,107820,
        38336,30000,107820,14500,40500,41930,250,936,14000,14400
    ]
}

df = pd.DataFrame(data)

pivot = pd.pivot_table(
    df,
    values="Sale_amt",
    index="Item",
    aggfunc=["max", "min"]
)

print(pivot)
```

OUTPUT

	max	min
	Sale_amt	Sale_amt
Item		
Cell Phone	14400	6075
Desk	250	250
Home Theater	40500	14000
Television	113810	38336
Video Games	936	936

Experiment 8 – Pivot Table for Item-wise Units Sold

AIM

To create a Pandas pivot table and find the total units sold for each item.

ALGORITHM

1. Import Pandas.

2. Create the item and units data.

3. Create a pivot table with Item as the index.

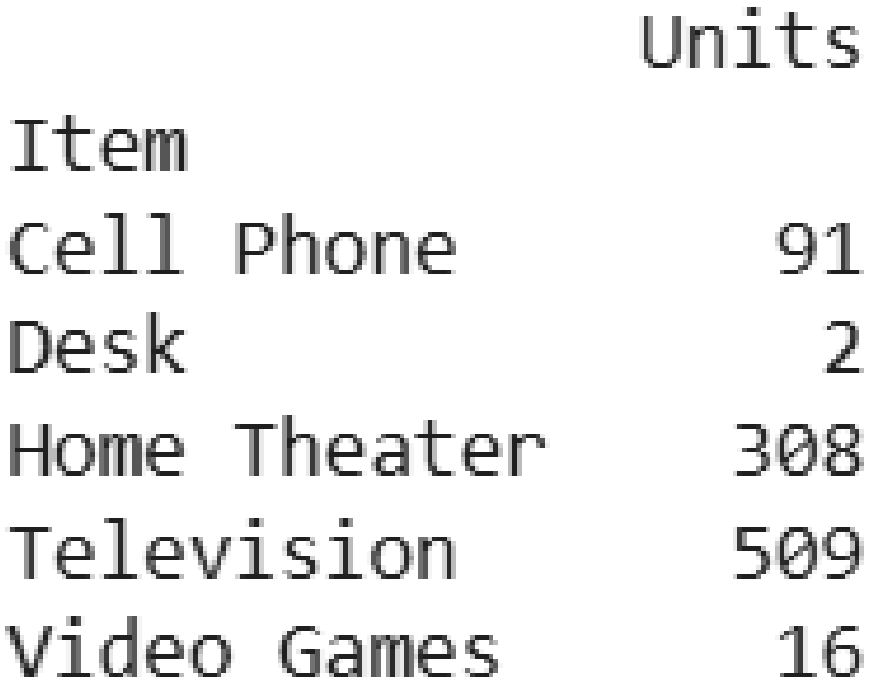
4. Use sum as the aggregation function for Units.

5. Display the item-wise total units sold.

CODE

```
import pandas as pd

data = {
    "Item": [
        "Television","Home Theater","Television","Cell Phone",
        "Television","Home Theater","Television","Television",
        "Television","Home Theater","Television","Home Theater",
        "Home Theater","Television","Desk","Video Games",
        "Home Theater","Cell Phone"
    ],
    "Units": [
        95,50,36,27,56,60,75,90,32,60,90,29,81,35,2,16,28,64
    ]
}

df = pd.DataFrame(data)

pivot = pd.pivot_table(
    df,
    values="Units",
    index="Item",
    aggfunc="sum"
)

print(pivot)
```

OUTPUT



Item	Units
Cell Phone	91
Desk	2
Home Theater	308
Television	509
Video Games	16

Experiment 9 – Total Sale Amount Region-wise, Manager-wise and Salesman-wise

AIM

To create a pivot table to find the total sale amount region-wise, manager-wise and salesman-wise.

ALGORITHM

1. Import Pandas.

2. Create the Sales_data DataFrame.

3. Select Region, Manager, SalesMan and Sale_amt.

4. Create a pivot table using the three grouping columns as the index.

5. Use sum as the aggregation function.

6. Display the total sale amount.

CODE

```
import pandas as pd

data = {
    "Region": [
        "East","Central","Central","Central","West","East",
        "Central","Central","West","East","Central","East",
        "East","East","Central","East","Central","East"
    ],
    "Manager": [
        "Martha","Hermann","Hermann","Timothy","Timothy","Martha",
        "Martha","Hermann","Douglas","Martha","Hermann","Martha",
        "Douglas","Martha","Douglas","Martha","Hermann","Martha"
    ],
    "SalesMan": [
        "Alexander","Shelli","Luis","David","Stephen","Alexander",
        "Steven","Luis","Michael","Alexander","Sigal","Diana",
        "Karen","Alexander","John","Alexander","Sigal","Alexander"
    ],
    "Sale_amt": [
        113810,25000,43128,6075,67088,30000,89850,107820,
        38336,30000,107820,14500,40500,41930,250,936,14000,14400
    ]
}

df = pd.DataFrame(data)

pivot = pd.pivot_table(
    df,
    values="Sale_amt",
    index=["Region","Manager","SalesMan"],
    aggfunc="sum"
)

print(pivot)
```

OUTPUT

Region	Manager	SalesMan	Sale_amt
Central	Douglas	John	250
	Hermann	Luis	150948
		Shelli	25000
		Sigal	121820
	Martha	Steven	89850
	Timothy	David	6075
East	Douglas	Karen	40500
	Martha	Alexander	231076
		Diana	14500
West	Douglas	Michael	38336
	Timothy	Stephen	67088

Experiment 10 – Highlight Negative and Positive Numbers

AIM

To create a 10-row, 4-column random DataFrame and highlight negative numbers in red and positive numbers in black.

ALGORITHM

1. Import Pandas and NumPy.

2. Generate a 10 × 4 DataFrame with random values.

3. Define a function to check each value.

4. Return red text for negative values and black text for positive values.

5. Apply the function using DataFrame styling.

6. Display the styled DataFrame.

CODE

```
import pandas as pd
import numpy as np

df = pd.DataFrame(
    np.random.randn(10, 4),
    columns=["A", "B", "C", "D"]
)

def highlight_numbers(value):
    if value < 0:
        return "color: red"
    else:
        return "color: black"

df.style.map(highlight_numbers)
```

OUTPUT

	A	B	C	D
0	-0.852493	0.530255	-0.240047	-1.066707
1	-1.484421	-0.190302	1.524237	-0.166135
2	0.524373	-0.052886	1.051071	0.911287
3	-1.251641	-1.178279	-1.712902	-0.228642
4	-2.187164	-0.317702	-0.461587	-1.174452
5	1.405396	-0.331198	0.382942	1.843226
6	-0.226064	-1.143635	1.587824	1.040012
7	0.355097	0.406998	1.507851	0.149209
8	-0.823213	0.062179	-0.351701	-0.554032
9	-0.735723	-0.588401	0.471206	0.858363